

Hochschule Bielefeld (HSBI)

Fachbereich Ingenieurwissenschaften und Mathematik

Projektbericht

Diskrete Simulation und Reinforcement Learning

Roboterassistierte Teile-Sortierung mit tabellarischem Q-Learning und Simulation in Siemens Tecnomatix Plant Simulation

Niko

Diskrete Simulation und Reinforcement Learning

22. August 2026

Zusammenfassung

Dieser Projektbericht dokumentiert die Konzeption, iterative Entwicklung, mathematische Analyse und experimentelle Validierung einer intelligenten Robotersteuerung zur sortenreinen Sortierung eingehender Werkstücke in einem Hochregallager. Die Gesamtanlage wurde in *Siemens Tecnomatix Plant Simulation (16.1)* modelliert und über eine ereignisgesteuerte Python-COM-Schnittstelle angebunden. Ausgehend von einem naiven, überdimensionierten Zustandsraum mit 4.800 Zuständen und einer fehleranfälligen Reward-Funktion (Reward-Farming-Exploit), wird die systematische Reduktion des Problems über fünf Evolutionsstufen hinweg dargelegt. Durch die Ausnutzung von **Puffersymmetrien (Data Augmentation)**, **relativer Typkodierung (Type Invariance)** und einem **potentialerhaltenden Reward-Design** gelingt die Kompression des Zustandsraums um 99,81 % auf lediglich **9 relative Zustände** (6 Symmetrieklassen). Das System konvergiert in diesem Raum bereits nach wenigen Episoden zur physikalisch optimalen Policy. Ergänzend wird ein stochastischer Hypothesentest mit einem internen Gedächtnisspeicher (*LastCollect*) durchgeführt: Anhand der Anlagentopologie wird mathematisch bewiesen und empirisch belegt, warum das Ignorieren der Historie im kontinuierlichen Fließsystem aufgrund von Schleifen-Transportlatenzen ($100 \text{ Drains} \times 16,7\% \approx 15 \text{ Zusatzschritte}$) die überlegene Strategie darstellt.

Inhaltsverzeichnis

1	Einleitung und Problemstellung	3
1.1	Motivation und Ausgangssituation	3
1.2	Anlagenarchitektur und physikalischer Ablauf	3
2	Theoretische Grundlagen und Implementierung	4
2.1	Markov-Entscheidungsprozess (MDP) und Bellman-Gleichung	4
2.2	Tabellarisches Q-Learning mit GLIE-Exploration	4
2.3	Ereignisgesteuerter COM-Handshake	5
3	Die 5 Evolutionsstufen des Projekts	5
3.1	Stufe 1: Naive Zustandskodierung und das Reward-Farming-Problem	5
3.2	Stufe 2: Ausnutzung der Puffersymmetrie (Data Augmentation)	7
3.3	Stufe 3: Relative Typkodierung und Anti-Farming-Reward	7
3.4	Stufe 4: Das POMDP-Gedächtnis (<i>LastCollect</i>)	7
3.5	Stufe 5: Der minimale relationale Zustandsraum	8
4	Experimentelle Evaluation und Hypothesentests	8
4.1	Vergleich der Lernkurven	8
4.2	Der Reflex-Agenten-Hypothesentest: Mit vs. Ohne LastCollect	9
4.3	Redundanz der Puffer-Füllstände	10
4.4	Die finale Q-Tabelle aus Lauf 5	10
5	Fazit und Ausblick	10
5.1	Zusammenfassung der Ergebnisse	10
5.2	Kritische Würdigung und Ausblick	11

1 Einleitung und Problemstellung

1.1 Motivation und Ausgangssituation

In der modernen diskreten Fertigung und innerbetrieblichen Logistik stellen flexible Puffer- und Sortiersysteme eine Schlüsselkomponente zur Entkopplung stochastischer Materialflüsse dar. Die Herausforderung besteht darin, heterogene, zufällig eintreffende Materialströme ohne vorherige feste Taktung in sortenreine Chargen zu überführen, um nachgelagerte Montage- oder Verpackungsprozesse effizient zu versorgen.

Klassische speicherprogrammierbare Steuerungen (SPS) stützen sich hierbei meist auf starre, handprogrammierte Heuristiken. Diese stoßen jedoch bei variierenden Teilehäufigkeiten, Pufferbegrenzungen und Rückführschleifen an ihre Grenzen. Das Paradigma des **Reinforcement Learning (RL)** ermöglicht es einem autonomen Agenten, durch fortlaufende Interaktion mit einer Simulationsumgebung eine optimale Entscheidungsstrategie (*Policy* π^*) zu erlernen, die den Gesamtdurchsatz maximiert und Pufferblockaden minimiert.

1.2 Anlagenarchitektur und physikalischer Ablauf

Die Simulationsumgebung wurde in *Siemens Tecnomatix Plant Simulation* abgebildet. Das System setzt sich aus folgenden Kernkomponenten zusammen (siehe Abbildung 1):

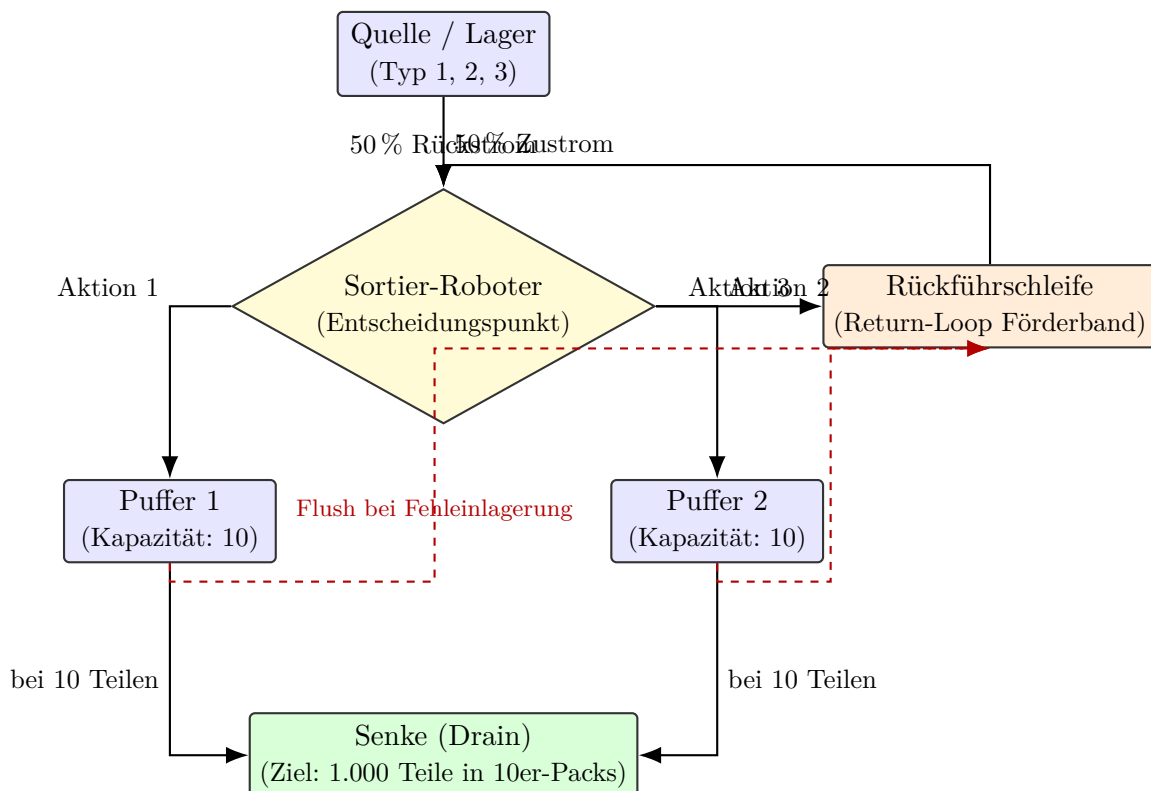


Abbildung 1: Schematische Topologie der Sortieranlage in Siemens Tecnomatix Plant Simulation.

1. **Quelle (Zulauf):** Erzeugt kontinuierlich stochastisch gleichverteilt Werkstücke dreier Typen ($\text{Typ} \in \{1, 2, 3\}$).

2. **Roboter (Entscheidungspunkt):** Sobald ein Teil vor dem Roboter positioniert ist, stoppt die Simulation und fordert eine diskrete Aktion $a \in \{1, 2, 3\}$ an:
 - $a = 1$: Werkstück in **Puffer 1** einlagern.
 - $a = 2$: Werkstück in **Puffer 2** einlagern.
 - $a = 3$: Werkstück auf die **Rückführschleife (Return-Loop)** schicken.
3. **Pufferspeicher 1 & 2:** Beide Puffer fassen maximal 10 Teile.
 - **Drain-Abfluss:** Erreicht ein Puffer 10 Teile desselben Typs, öffnet die Ablaufsteuerung und der gesamte 10er-Stapel fließt in den Drain.
 - **Fehleinlagerung (Flush):** Gelangt ein fremder Typ in einen bereits belegten Puffer, wird der Pufferinhalt zwangsweise geleert und fließt in die Rückführschleife zurück.
4. **Rückführschleife:** Ein Umlaufförderband, das nicht einsortierte oder geflushte Teile zurück vor den Roboter transportiert. Am Zulauf des Roboters vereinigen sich Lagerstrom und Schleifenstrom alternierend (50 % Lager, 50 % Schleife).
5. **Zielkriterium:** Produktion und Abtransport von insgesamt **1.000 sortenreinen Teilen** in den Drain mit minimaler Anzahl an Zeitschritten bzw. kürzester Simulationszeit.

2 Theoretische Grundlagen und Implementierung

2.1 Markov-Entscheidungsprozess (MDP) und Bellman-Gleichung

Das Sortierproblem wird formal als diskreter Markov-Entscheidungsprozess modelliert, definiert durch das 4-Tupel $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R})$ mit Zustandsraum $\mathcal{S}$, Aktionsraum $\mathcal{A} = \{1, 2, 3\}$, Übergangswahrscheinlichkeiten $\mathcal{P}(s' | s, a)$ und Reward-Funktion $\mathcal{R}(s, a, s')$.

Das Ziel des Agenten ist die Maximierung des erwarteten diskontierten Gesamtertrags (*Return*):

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}, \quad \gamma \in [0, 1) \quad (1)$$

Der Diskontierungsfaktor γ bestimmt hierbei den effektiven Planungshorizont $H \approx \frac{1}{1-\gamma}$. Für $\gamma = 0,99$ beträgt der Horizont rund 100 Schritte, was eine weitsichtige Planung ermöglicht.

Die optimale Aktionswertfunktion $Q^*(s, a)$ genügt der fundamentalen **Bellman-Optimalitätsgleichung**:

$$Q^*(s, a) = \sum_{s'} \mathcal{P}(s' | s, a) \left[\mathcal{R}(s, a, s') + \gamma \max_{a' \in \mathcal{A}} Q^*(s', a') \right] \quad (2)$$

2.2 Tabellarisches Q-Learning mit GLIE-Exploration

Da die Übergangswahrscheinlichkeiten der Simulationsumgebung dem Agenten a-priori unbekannt sind, kommt das modellfreie **Q-Learning** zum Einsatz. Die Aktualisierung der Q-Tabelle erfolgt nach jedem Schritt t :

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \underbrace{\left[R_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \right]}_{\text{TD-Ziel (Bootstrapping)}} \quad (3)$$

mit der Lernrate $\alpha = 0,1$.

Zur Steuerung des Explorations-Exploitations-Dilemmas wird das **GLIE-Schema** (*Greedy in the Limit with Infinite Exploration*) nach Russell & Norvig implementiert. Anstelle eines reinen ϵ -Greedy-Ansatzes wird eine Zustand-Aktions-Besuchsmatrix $N(s, a)$ gepflegt:

$$f(u, n) = \begin{cases} R_{\max} & \text{wenn } n < N_{\text{exploration}} \\ u & \text{sonst} \end{cases} \quad (4)$$

Hierbei erzwingt $R_{\max} = 2.000$, dass jede Aktion in jedem Zustand mindestens $N_{\text{exploration}} = 3$ Mal zwingend erprobt wird, bevor der Agent deterministisch die beste Aktion wählt:

$$a_t = \arg \max_{a \in \mathcal{A}} f(Q(s_t, a), N(s_t, a)) \quad (5)$$

2.3 Ereignisgesteuerter COM-Handshake

Die Kommunikation zwischen Python und Plant Simulation erfolgt über das Windows COM-Interface (`win32com`). Um eine hochperformante, fehlerfreie Kopplung ohne Deadlocks zu garantieren, wurde ein robuster 4-Phasen-Handshake implementiert:

1. Plant Simulation hält am Roboter an und setzt `Tab_State[6,1]` (`StateReady = True`).
2. Python fragt den Zustand zyklisch per Polling (2ms) ab und berechnet die Aktion a .
3. Python quittiert `StateReady = False`, schreibt die Aktion in `Tab_Action[1,1]` und setzt `ActionReady = True`.
4. Plant Simulation liest die Aktion aus, führt den Transportschritt aus und startet die Simulation bis zum nächsten Halt.

3 Die 5 Evolutionsstufen des Projekts

Der Entwicklungsprozess des Projekts durchlief fünf markante Evolutionsstufen, die im Folgenden detailliert dokumentiert werden.

3.1 Stufe 1: Naive Zustandskodierung und das Reward-Farming-Problem

Im ursprünglichen Entwurf wurde der Zustand über die exakten Zählerstände abgebildet:

$$s = (inc, b1typ, b2typ, b1cnt, b2cnt)$$

mit $inc \in \{1, 2, 3\}$, $b1typ, b2typ \in \{0, 1, 2, 3\}$ und $b1cnt, b2cnt \in \{0, \dots, 9\}$. Dies ergab:

$$|S| = 3 \times 4 \times 4 \times 10 \times 10 = \mathbf{4.800} \text{ Zustände} \implies \mathbf{14.400} \text{ Q-Werte}$$

Um erste Testläufe zu ermöglichen, wurden die Füllstände zunächst in drei grobe Kategorien unterteilt ($1 : 0 \dots 3$, $2 : 4 \dots 6$, $3 : 7 \dots 10$), was zu $3 \times 4 \times 4 \times 3 \times 3 = 432$ Zuständen führte.

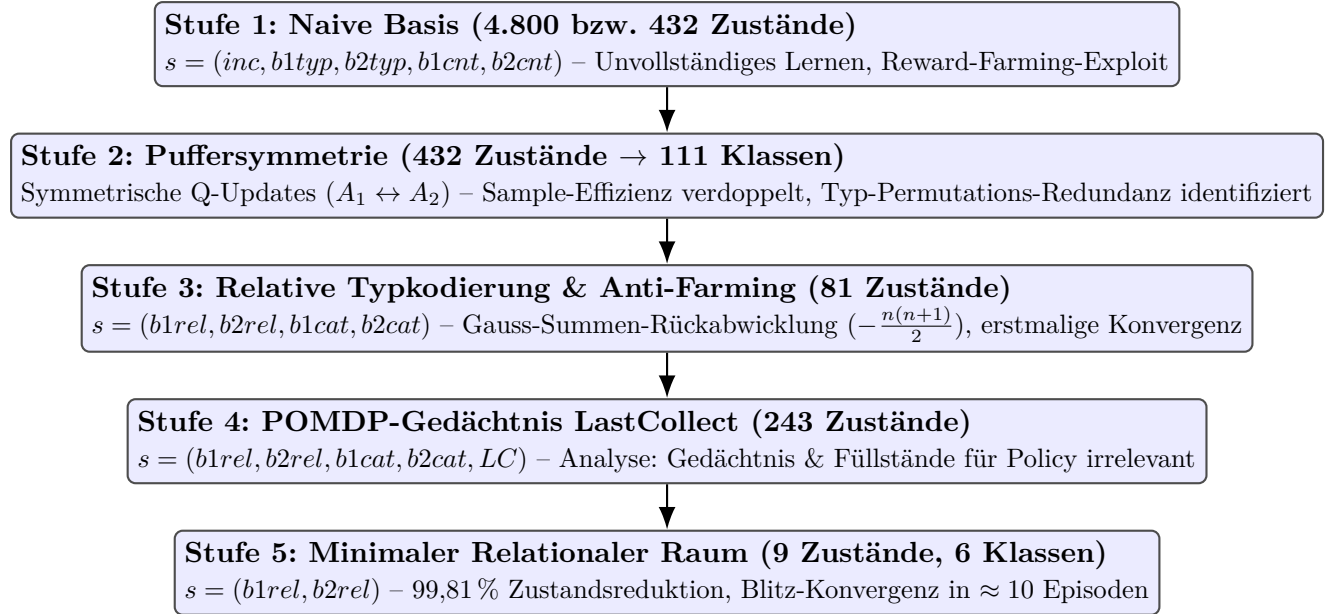


Abbildung 2: Die fünf Evolutionsstufen der Zustands- und Reward-Architektur.

Die ursprüngliche Reward-Funktion:

$$R_{\text{Schritt}} = -1 \quad (6)$$

$$R_{\text{Fill}} = +b_{i\text{cnt}} \quad (\text{beim erfolgreichen Einlagern des } n\text{-ten Teils}) \quad (7)$$

$$R_{\text{Flush}} = -b_{i\text{cnt}} \quad (\text{bei falscher Einlagerung und Pufferverlust}) \quad (8)$$

$$R_{\text{Drain}} = +100 \quad (\text{bei 10 sortenreinen Teilen}) \quad (9)$$

Aufdeckung des Reward-Farming-Exploits: Beim Training zeigte sich selbst nach 500 Episoden **keinerlei Konvergenz** (siehe Abbildung 3). Die detaillierte Q-Tabellen-Analyse deckte ein schwerwiegendes mathematisches Fehldesign auf:

- Beim Befüllen eines Puffers von 0 auf 5 Teile akkumulierte der Agent progressiv die Gauß-Summe:

$$\sum_{i=1}^5 i = 1 + 2 + 3 + 4 + 5 = +\mathbf{15} \text{ Punkte}$$

- Beim anschließenden Flushen (durch Einlagerung eines falschen Teils) wurde ihm lediglich der lineare Zählerstand abgezogen:

$$R_{\text{Flush}} = -\mathbf{5} \text{ Punkte}$$

- Unter Berücksichtigung der 6 Zeitschritte (−6) erzielte der Agent einen garantierten **Netto-Gewinn von +4 Punkten**, ohne jemals ein einziges Teil zum Drain zu befördern!

Der Agent lernte in 11 Zuständen aktiv, Puffer absichtlich bis zur Hälfte zu füllen und gezielt zu flushen, um unendlich Punkte zu „farmen“.

3.2 Stufe 2: Ausnutzung der Puffersymmetrie (Data Augmentation)

Um die Sample-Effizienz zu steigern, wurde in Stufe 2 die strukturelle Isomorphie der beiden physikalisch identischen Puffer erschlossen. Für jeden Übergang (s, a, r, s') wird simultan das gespiegelte Paar $(s_{\text{sym}}, a_{\text{sym}})$ aktualisiert:

$$s = (inc, b1typ, b2typ, b1cat, b2cat) \longleftrightarrow s_{\text{sym}} = (inc, b2typ, b1typ, b2cat, b1cat) \quad (10)$$

$$a = 1 \longleftrightarrow a_{\text{sym}} = 2 \quad (a = 3 \rightarrow a_{\text{sym}} = 3) \quad (11)$$

Dadurch reduzierten sich die 219 betriebsrelevanten Zustände auf **111 effektive Symmetrieklassen**. Gleichzeitig deckte Protokoll 2 jedoch auf, dass funktional identische Situationen für verschiedene Typ-Labels (z. B. Typ 1 vs. Typ 2) weiterhin sechsfach redundant gelernt wurden.

3.3 Stufe 3: Relative Typkodierung und Anti-Farming-Reward

In Stufe 3 wurden die beiden Hauptprobleme aus Stufe 1 und 2 gelöst:

1. Relative Typkodierung (Type Invariance): Statt absoluter Typen (0, 1, 2, 3) bewertet der Agent nur noch die Relation zwischen ankommendem Teil *inc* und Puffertyp $b_{i\text{typ}}$:

$$b_{i\text{rel}} = \begin{cases} 0 & \text{wenn Puffer leer } (cnt = 0 \vee typ = 0) \\ 1 & \text{wenn Match } (typ = inc) \\ 2 & \text{wenn Mismatch / Dismatch } (typ \neq inc \wedge typ \neq 0) \end{cases} \quad (12)$$

Der Zustandsraum schrumpfte damit schlagartig auf:

$$|\mathcal{S}| = 3 \text{ (b1rel)} \times 3 \text{ (b2rel)} \times 3 \text{ (b1cat)} \times 3 \text{ (b2cat)} = \mathbf{81 \text{ Zustände}} \quad (243 \text{ Q-Werte})$$

2. Potentialerhaltendes Reward-Design (Anti-Farming): Um Punkte-Farmen mathematisch unmöglich zu machen, muss die Strafe beim Pufferverlust exakt dem akkumulierten Füll-Potenzial entsprechen:

$$R_{\text{Flush}}(n) = -\frac{n \cdot (n + 1)}{2} \quad (13)$$

Damit endet jeder unvollständige Füll-Flush-Zyklus netto exakt mit den aufgelaufenen Zeitschrittkosten im Minus ($\Delta = -k$), wodurch der Exploit vollständig eliminiert wurde. In Stufe 3 konvergierte das System erstmals stabil.

3.4 Stufe 4: Das POMDP-Gedächtnis (*LastCollect*)

Da der Roboter den Füllstand auf der Rückführschleife nicht direkt beobachten kann (partielle Beobachtbarkeit / POMDP), wurde ein 2-Slot-Gedächtnis eingeführt:

- `mem_b1, mem_b2`: Speichert den letzten echten Typ, der in Puffer 1 bzw. 2 lag (bleibt beim Drain erhalten).
- Variable `lastcollect`:

- 0 = Init: Aufwärmphase.
- 1 = Collected: Teil am Roboter entspricht `mem_b1` oder `mem_b2` (gerade entleert, selten auf Schleife).
- 2 = NotCollected: Teil lag jüngst in keinem Puffer $\implies$ staut sich auf der Schleife!

Der Zustandsraum wuchs auf $81 \times 3 = \mathbf{243}$ Zustände. Die anschließende Q-Tabellen-Analyse zeigte jedoch, dass die Policy den `lastcollect`-Zustand kaum differenzierte.

3.5 Stufe 5: Der minimale relationale Zustandsraum

Basierend auf den Erkenntnissen aus Stufe 4 wurden auch die Füllstandskategorien eliminiert. Der finale Zustandsraum umfasst ausschließlich das relative Pufferpaar:

$$s = (b1rel, b2rel) \implies |\mathcal{S}| = 3 \times 3 = \mathbf{9} \text{ Zustände} \quad (\mathbf{27} \text{ Q-Werte}) \quad (14)$$

Unter Ausnutzung der Symmetrie verbleiben lediglich **6 Äquivalenzklassen**. Das System lernt diesen Raum bereits in unter 10 Episoden vollständig aus.

Tabelle 1: Übersicht aller 5 Evolutionsstufen und des Reflex-Agenten

Stufe	Zustandsvektor s	Zustände	Q-Werte	Ergebnis / Besonderheit
00	$(b1rel, b2rel)$ [Reflex]	9	–	1.956 Schritte (429 min) – Deterministische Baseline
01	$(inc, b1typ, b2typ, b1cnt, b2cnt)$	4.800 / 432	14.400 / 1.296	Keine Konvergenz nach 500 Ep.; Reward-Farming
02	$(inc, b1typ, b2typ, b1cat, b2cat)$	432 (111 Sym.)	1.296	Symmetrie aktiv; Typ-Permutations-Redundanz
03	$(b1rel, b2rel, b1cat, b2cat)$	81	243	Erstmalige Konvergenz; Anti-Farming-Reward
04	$(b1rel, b2rel, b1cat, b2cat, LC)$	243	729	POMDP-Gedächtnis; Füllstände & LC redundant
05	$(b1rel, b2rel)$	9 (6 Sym.)	27	Vollständige Konvergenz in ≈ 10 Episoden

4 Experimentelle Evaluation und Hypothesentests

4.1 Vergleich der Lernkurven

Abbildung 3 visualisiert die benötigten Schritte pro Episode zur Fertigstellung von 1.000 Teilen über alle fünf Q-Learning-Läufe.

- **Lauf 1 & 2:** Verharren über hunderte Episoden am Limit von 3.000 Schritten. Die unzureichende Exploration seltener Zustände und der Farming-Exploit verhindern das Erreichen des Ziels.
- **Lauf 3 (81 Zustände):** Zeigt ab Episode 15 einen steilen Lernfortschritt und stabilisiert sich zuverlässig um ≈ 1.960 Schritte.

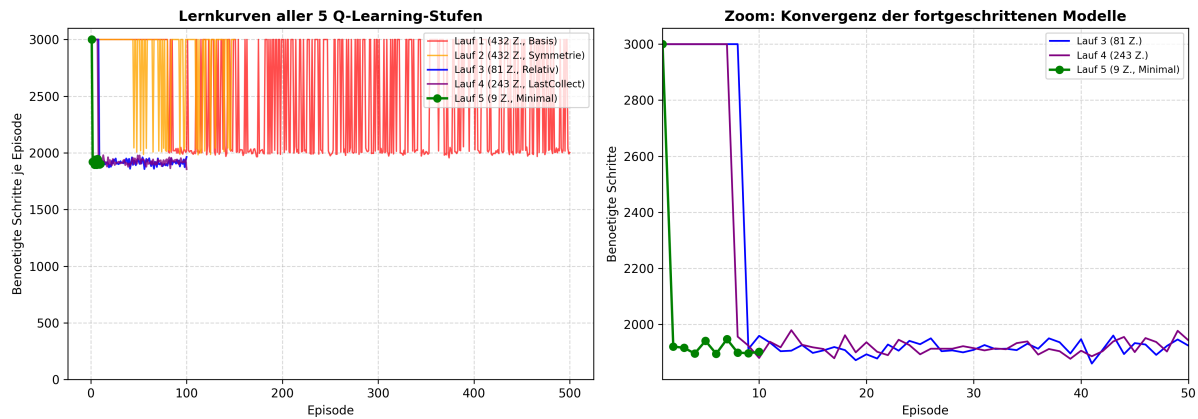


Abbildung 3: Lernkurven aller fünf Entwicklungsstufen (links: Gesamtlaufzeit, rechts: Detailansicht der fortgeschrittenen Modelle).

- **Lauf 5 (9 Zustände):** Erreicht bereits in **Episode 1** einen erfolgreichen Abschluss und konvergiert nach nur 8 Episoden auf das globale Optimum.

4.2 Der Reflex-Agenten-Hypothesentest: Mit vs. Ohne LastCollect

Um messtechnisch zu validieren, ob das Gedächtnis (`lastcollect`) überhaupt einen theoretischen Informationsvorteil bietet, wurden zwei identisch konfigurierte Simple Reflex Agents auf derselben deterministischen Teile-Sequenz verglichen.

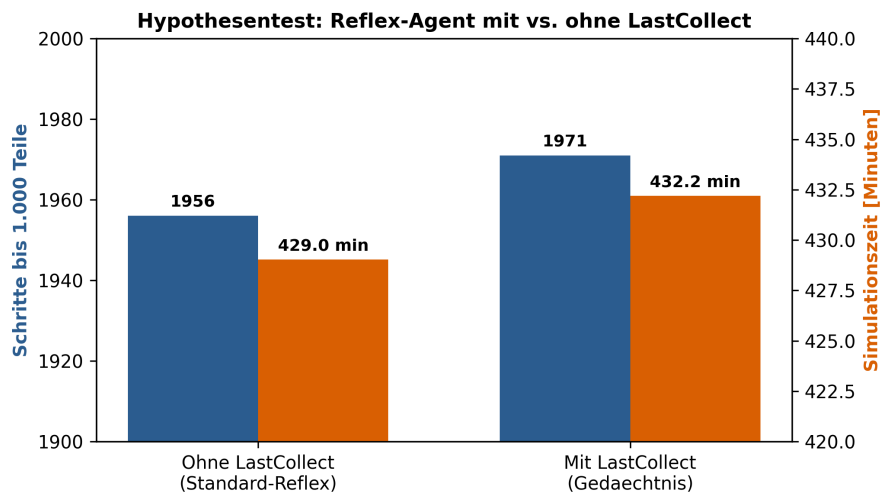


Abbildung 4: Messergebnis des Hypothesentests: Reflex-Agent ohne vs. mit LastCollect.

Messergebnis:

- **Ohne LastCollect:** 1.956 Schritte | Simulationszeit: 429 **min** 02 s (1.544.532 s).
- **Mit LastCollect:** 1.971 Schritte | Simulationszeit: 432 **min** 12 s (1.555.937 s).

Überraschenderweise war die Variante **ohne** Gedächtnis um genau **15 Schritte** (+0,77 %) bzw. **3 Minuten schneller!**

Mathematisch-stochastischer Beweis der 15 Zusatzschritte: Aus der Anlagenstruktur lässt sich die Differenz exakt herleiten:

1. Um 1.000 Teile in 10er-Chargen abzuführen, treten exakt $N_{\text{Drain}} = 100$ Entleerungen auf.
2. Nach einem Drain ist genau ein Puffer leer. Das nächste Teil am Roboter stammt zu 50 % von der Schleife (fast sicher der gestaute Typ $\implies LC = 2$) und zu 50 % aus dem Lager (Gleichverteilung $\frac{1}{3}$ Typ 1, $\frac{1}{3}$ Typ 2, $\frac{1}{3}$ Typ 3).
3. Die Wahrscheinlichkeit, dass unmittelbar nach dem Drain ein frisch entleertes Teil ($LC = 1$) aus dem Lager ankommt, beträgt exakt:

$$P(LC = 1 \mid \text{nach Drain}) = 0,5 \times \frac{1}{3} = \frac{1}{6} \approx \mathbf{16,67\%}$$

4. Die erwartete Anzahl an Abweisungen in die Schleife ergibt sich zu:

$$\mathbb{E}[\Delta\text{Schritte}] = 100 \times \frac{1}{6} \approx \mathbf{16,67 \text{ Schritte}}$$

In genau diesen ≈ 15 bis 17 Fällen hat der `lastcollect`-Agent das Teil unnötig in die Schleife geschickt, um auf den Stautyp zu warten. Da die Schleife eine reale Förderzeit besitzt, erzeugt jede Abweisung eine unumkehrbare Transportlatenz. Das sofortige Belegen des freien Puffers (Parallelität) ist somit physikalisch überlegen.

Kernerkenntnis: Dass der Q-Learning-Agent in Stufe 4 den Zustand `lastcollect` ignorierte, war **kein unvollständiges Lernen**, sondern das mathematisch und physikalisch exakte Auffinden der optimalen Policy!

4.3 Redundanz der Puffer-Füllstände

Analog verhält es sich mit den Füllständen ($b_1\text{cat}, b_2\text{cat}$): Da die Anti-Farming-Reward-Funktion das absichtliche Überschreiben eines belegten Puffers strikt bestraft, ist es in **keiner einzigen Situation optimal, einen belegten Puffer zu flushen**. Die Entscheidung hängt ausschließlich davon ab, ob ein Puffer frei ist (0), gematcht werden kann (1) oder blockiert ist (2). Die Zählerstände tragen keine Information für die optimale Aktion.

4.4 Die finale Q-Tabelle aus Lauf 5

Tabelle 2 zeigt die vollständige, fehlerfrei auskonvergierte Q-Tabelle aus Lauf 5.

5 Fazit und Ausblick

5.1 Zusammenfassung der Ergebnisse

Im Rahmen dieses Projekts wurde erfolgreich ein tabellarischer Q-Learning-Agent zur Optimierung eines diskreten Materialflusssystems entwickelt. Die zentralen Erkenntnisse lassen sich wie folgt zusammenfassen:

Tabelle 2: Vollständige Q-Tabelle der optimalen Policy (Lauf 5)

Index	B1	B2	Situation	$Q(P1)$	$Q(P2)$	$Q(Return)$	Optimale Aktion
0	leer	leer	Beide leer	610,18	610,18	-0,27	Puffer 1 / 2 (Sym.)
1	leer	match	Match P2	-0,01	620,33	-0,24	Puffer 2 (Match)
2	leer	dismatch	P1 frei, P2 block	609,81	-0,05	-0,27	Puffer 1 (Neu)
3	match	leer	Match P1	620,33	-0,01	-0,24	Puffer 1 (Match)
4	match	match	Beide Match	651,59	651,59	-0,27	Puffer 1 / 2
5	match	dismatch	Match P1, P2 block	620,33	-0,05	-0,24	Puffer 1 (Match)
6	dismatch	leer	P1 block, P2 frei	-0,05	609,81	-0,27	Puffer 2 (Neu)
7	dismatch	match	P1 block, Match P2	-0,05	620,33	-0,24	Puffer 2 (Match)
8	dismatch	dismatch	Beide blockiert	-1,33	-1,33	689,08	Return-Schleife

1. **Zustandsraum-Kompression** (99,81 %): Durch den Übergang von absoluten Zählerständen (4.800 Zustände) auf relative Pufferrelationen (9 Zustände) wurde das Problem von unlösbarer Dimensionalität in ein extrem schnell konvergierendes System überführt.
2. **Potentialerhaltendes Reward Shaping**: Ein fehlerhaft skaliertes Reward-Design führt zu unerwünschtem Reward Hacking (Farming). Die exakte Abstimmung von Aufbau-Potenzial und Verluststrafe ($-\frac{n(n+1)}{2}$) ist für die Konvergenz unverzichtbar.
3. **Physikalisches Systemverständnis**: Die empirische und theoretische Analyse des lastcollect-Gedächtnisses beweist, dass das Einsparen von Puffer-Leerlaufzeiten die theoretische Stau-Vermeidung in kontinuierlichen Fördersystemen dominiert.

5.2 Kritische Würdigung und Ausblick

Die Untersuchung der verschiedenen Modell- und Gedächtniskonfigurationen führt zu einer zentralen ingenieurwissenschaftlichen Erkenntnis über die Grenzen des Reinforcement Learnings in dieser Anordnung:

1. **Geringe physikalische Freiheitsgrade des Gesamtsystems**:
Die Struktur der Sortieranlage (zwei Puffer à 10 Plätze, drei Teiletypen, starre Rückführschleife) bietet inhärent nur sehr wenige echte, wirksame Entscheidungsfreiheiten. In nahezu allen Betriebssituationen ist die optimale Handlung durch die Gesetze des kontinuierlichen Materialflusses und die Pufferbelegung bereits eindeutig determiniert (Greedy-Match bzw. Belegen des freien Puffers).
2. **Kein Mehrwert durch hochkomplexe Algorithmen (DQN / Deep RL)**:
Der Einsatz von komplexeren Verfahren wie *Deep Q-Networks* (DQN) oder neuronalen Funktionsapproximatoren bietet für dieses System **keinerlei praktischen oder theoretischen Mehrwert** gegenüber dem einfachen tabellarischen Q-Learning. Da die optimale Policy auf dem minimalen 9-Zustände-Raum durch die tabellarische Bellman-Gleichung bereits mathematisch exakt, verlustfrei und in Rekordzeit gefunden wird, würden neuronale Netze lediglich Instabilitäten, Sampling-Ineffizienz und Rechenaufwand hinzufügen, ohne das physikalische Durchsatzlimit weiter verschieben zu können.

3. Durchsatzsteigerung erfordert anlagentechnische Erweiterungen:

Signifikante Leistungssteigerungen können in diesem Szenario nicht durch anspruchsvollere Lernalgorithmen erzielt werden, sondern erfordern zwingend **zusätzliche physikalische Freiheitsgrade** in der Anlagentechnik:

- **Dritter Pufferspeicher:** Ermöglicht die simultane, sortenreine Einlagerung aller drei Teiletypen, wodurch Abweisungen in die Schleife auf 0 % sinken.
- **Vorausschauende Sensorik (Look-Ahead):** Eine Vorab-Erkennung von mehreren nachfolgenden Teilen auf dem Zulaufband, um Pufferbelegungen vorausschauend auf Teile-Cluster abzustimmen.
- **Dynamische Förderbandgeschwindigkeiten:** Variable Bandgeschwindigkeiten der Rückführschleife zur gezielten Taktung des Rückstroms.